

Lab 3: Architectural Justification

Scenario #41: OpenAPI Mock Server Generator

Architecture Selection

We chose **Layered Architecture** for the **OpenAPI Mock Server Generator** System.

Architectural Choice

We selected the Layered style with three layers. The **Presentation Layer** holds the Mock Endpoint Handler (REST entry point for spec uploads and mock requests). The **Business Layer** holds the Spec Parser, Schema Validator, Mock Response Generator and Latency Simulator. The **Data Layer** holds the Spec Repository. Components communicate only through the interfaces `IParseSpec`, `IValidate`, `IGenerateResponse`, `ISimulateLatency` and `ISpecStore`.

Style	Fit for the OpenAPI Mock Server Generator	Verdict
Layered	Maps to entry point, mock logic and spec storage; in-process calls add no network jitter.	Selected
Microservices	Network hops between services add latency variance that threatens the ± 10 ms target; heavy for a single developer tool.	Rejected
Client-Server	Describes deployment more than internal structure; central server is a single point of failure.	Partial fit

Two Reasons

1. **Clear separation of concerns.** The system naturally splits into request handling, spec processing (parse, validate, generate, delay) and spec storage. Each maps to one layer, so a component such as the Mock Response Generator can be replaced or improved without touching the parser or the storage, because only the interface is shared.
2. **Right-sized for the scope.** The tool is a single deployable developer utility with five focused functions. Microservices would add service discovery, network calls and operational overhead without any scaling benefit, and Client-Server alone does not describe the internal structure of the mock engine.

Security Advantage

All uploads and requests enter through a single point, the Mock Endpoint Handler, and must pass the Spec Parser and Schema Validator before reaching storage or the response generator. This lets us enforce spec size limits, safe YAML parsing, refusal to resolve external `$ref` URLs (preventing SSRF), and strict schema checking in one place, which shrinks the attack surface.

Performance Benefit

All layers run in one process, so components interact through in-process calls with no network hops. Parsed specs are cached by the Spec Repository, so the per-request path (lookup, validate, generate, delay) stays short. This supports the target of 1,000 requests per second and keeps simulated latency accurate within ± 10 ms, because the only deliberate delay comes from the Latency Simulator.